Introduzione agli Algoritmi e Precisione Finita nel Calcolo Numerico

Stefania Bellavia, Greta Malaspina - Università di Firenze

Introduzione

Questo documento riassume i concetti fondamentali del corso di Calcolo Numerico per Ingegneria Informatica presso l'Università di Firenze, concentrandosi su algoritmi, rappresentazione dei numeri nei computer e problematiche legate alla precisione finita. Il calcolo numerico si occupa della risoluzione numerica di problemi matematici che non possono essere risolti analiticamente "con carta e penna".

Calcolo Numerico: Obiettivi e Approccio

Definizione e Scopo

Il calcolo numerico ha come obiettivo principale la **risoluzione numerica di problemi matematici complessi**. Il processo si articola in diverse fasi:

1. **Formulazione del problema matematico** (es. sistemi lineari $Ax = b$, integrali definiti $\int_a^b f(x)dx$)
2. **Sviluppo di un metodo numerico** che fornisce una soluzione esatta o approssimata (non analitica)
3. **Descrizione mediante algoritmo** del metodo risolutivo
4. **Implementazione** in un linguaggio di programmazione (nel corso si utilizza MATLAB)
5. **Esecuzione del programma** per calcolare un'approssimazione numerica della soluzione

Contenuti del Corso

Il corso tratta i seguenti argomenti principali:

- Algoritmi: istruzioni e strutture fondamentali
- Aritmetica finita: rappresentazione di numeri reali sul computer ed errori di arrotondamento
- Metodi numerici per:
  - Sistemi lineari
  - Approssimazione di dati e funzioni
  - Approssimazione di integrali definiti
  - Equazioni non lineari
- Introduzione al linguaggio MATLAB

Algoritmi: Definizione e Caratteristiche

Definizione Formale

Un **algoritmo** è:

- Una **successione finita di istruzioni**
- Assegnate in modo **non ambiguo**
- La cui esecuzione consente di passare da una situazione iniziale (DATI) a una situazione finale (RISULTATI)
- In un **tempo finito**

Esempio Pratico: Cucinare la Pasta

Il documento presenta un esempio interessante per illustrare il concetto di algoritmo:

Ricetta 1 (NON è un algoritmo):
"Quando l'acqua bolle aggiungere sale quanto basta" - l'espressione "quanto basta" è ambigua.
"Aspettare 10-12 minuti di cottura" - l'intervallo 10-12 minuti non è preciso.

Ricetta 2 (È un algoritmo):
"Aggiungere 2 cucchiaini di sale" - quantità precisa.
"Aspettare 12 minuti di cottura" - tempo definito.

Proprietà degli Algoritmi

Un buon algoritmo deve possedere:

- **Costo**: quantità di risorse necessarie (memoria, tempo di calcolo)
- **Generalità**: capacità di risolvere una classe di problemi (non solo un caso specifico)
- **Stabilità**: bassa sensibilità agli errori che si accumulano durante la risoluzione numerica

Requisiti Fondamentali

- **GENERALITÀ**: adatto a risolvere una classe di problemi
- **OTTIMALITÀ**: rispetto al tempo, al numero di operazioni, alla stabilità

Tipi di Errore nel Calcolo Numerico

Nel calcolo numerico si incontrano diversi tipi di errore:

Errori di Approssimazione

- **Discretizzazione**: trasformazione di un problema continuo in uno discreto (es. $\int_a^b f(x)dx \Rightarrow \sum_{i=0}^n w_i f(x_i)$)
- **Troncamento**: arresto di un processo iterativo prima della convergenza completa (es. metodi iterativi $\{x_k\} \to x^*$, si approssima x^* con $x_{\bar{k}}$ per un certo $\bar{k} > 0$)

Errori di Arrotondamento

Derivano dalla **precisione finita** con cui i computer rappresentano i numeri reali.

Esempi di Algoritmi Fondamentali

Somma di n Numeri

1. Leggi: $n, a_1, a_2, \ldots, a_n$
2. Poni $s = 0$
3. Per $i = 1, 2, \ldots, n$:
    - Poni $s = s + a_i$

4. Scrivi s
5. Stop

Prodotto di n Numeri

1. Leggi: $n, a_1, a_2, \ldots, a_n$
2. Poni $p = 1$
3. Per $i = 1, \ldots, n$:
    - Poni $p = p \times a_i$
4. Scrivi p
5. Stop

Valutazione di Polinomio

Dato il polinomio $P_n(x) = \sum_{i=0}^{n} a_i x^i$, si vuole calcolare $P_n(\alpha)$.

Algoritmo 1 (meno efficiente):

Operazioni necessarie:

- n moltiplicazioni
- n addizioni
- $\sum_{i=1}^{n}(i-1)$ moltiplicazioni per calcolare α^i

Algoritmo 2 (più efficiente):

Sfrutta il fatto che $\alpha^i = \alpha \times \alpha^{i-1}$, riducendo il numero totale di operazioni.

Operazioni necessarie:

- $2n$ moltiplicazioni
- n addizioni

Procedimento Iterativo: Calcolo di $\sqrt{N}$

Per calcolare $\sqrt{N}$ (con $N > 0$):

1. Individuare $m, n > 0$ tali che $m^2 < N < n^2$
2. Calcolare $c = \frac{m+n}{2}$
3. Se $c^2 = N \Rightarrow c = \sqrt{N}$ → Stop
4. Se $c^2 < N \Rightarrow$ Sostituire c a m e tornare al passo 2

5. Se $c^2 > N \Rightarrow$ Sostituire c a n e tornare al passo 2

Problema: Questo procedimento può non terminare (es. con $N = 2$), quindi **non è un algoritmo**.

Criterio di Arresto

Per rendere il procedimento un vero algoritmo, si introduce una **tolleranza** $\varepsilon > 0$.

Dato che $|c - \sqrt{N}| \leq \frac{1}{2}(n - m)$, si arresta il processo quando:

$$\frac{1}{2}(n - m) \leq \varepsilon$$

Così c diventa un'approssimazione di $\sqrt{N}$ entro la tolleranza scelta.

Rappresentazione dei Numeri nei Computer

Sistemi di Numerazione

I computer utilizzano rappresentazioni numeriche in basi diverse dal sistema decimale ($\beta = 10$):

- **Sistema Binario** ($\beta = 2$): cifre 0, 1
- **Sistema Decimale** ($\beta = 10$): cifre 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Notazione posizionale:

$(1721)_{10} = 1 \times 10^3 + 7 \times 10^2 + 2 \times 10 + 1$

$(1011)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2 + 1 = (11)_{10}$

$(3.21)_{10} = 3 \times 10^0 + 2 \times 10^{-1} + 1 \times 10^{-2}$

$(1.01)_2 = 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} = (1.25)_{10}$

Numeri Interi in Macchina

La rappresentazione degli interi utilizza:

- **1 bit per il segno** (0 $\Rightarrow N > 0$; 1 $\Rightarrow N < 0$)
- $t - 1$ **bits per la rappresentazione vera e propria**

Se il numero di bits disponibili non è sufficiente, si verifica **OVERFLOW**.

Numeri Reali: Virgola Mobile Normalizzata

Un numero reale x si rappresenta nella forma:

$$\pm a_0 . a_1 a_2 \ldots a_{m-1} a_m \ldots \beta^b$$

con $a_0 \neq 0$ (forma **normalizzata**).

Componenti:

- **Mantissa**: $a_0, a_1, a_2, \ldots, a_{m-1}, a_m, \ldots$
- **Caratteristica**: b (esponente)

Esempi ($\beta = 10$):

- $0.1471 \rightarrow 1.471 \times 10^{-1}$
- $37.1 \rightarrow 3.71 \times 10^{1}$
- $0.003 \rightarrow 3.0 \times 10^{-3}$

Rappresentazione in Macchina

La rappresentazione richiede bits per:

- **Segno** del numero
- **Mantissa** (determina la precisione)
- **Caratteristica** (determina l'ordine di grandezza)

Limiti:

- Il limite sulla **caratteristica** delimita l'ordine di grandezza dei dati → rischio di **OVERFLOW** o **UNDERFLOW**
- Il limite sulla **mantissa** caratterizza la **precisione**

Precisione di Macchina

Floating Point: fl(x)

Quando un numero reale x richiede più di m cifre di mantissa:

$$x = \pm a_0 . a_1 a_2 \dots a_{m-1} a_m \dots a_{m+i} \dots \beta^b$$

viene approssimato con $fl(x)$ (floating di x).

Metodi di Approssimazione

Troncamento:

$$fl(x) = \pm a_0 . a_1 a_2 \dots a_{m-1} \beta^b$$

Arrotondamento:

$$fl(x) = \begin{cases} \pm a_0 . a_1 a_2 \dots a_{m-1} \beta^b & \text{se } a_m < \beta/2 \\ \pm a_0 . a_1 a_2 \dots a_{m-1} \beta^b \pm \beta^{-(m-1)} \beta^b & \text{se } a_m \geq \beta/2 \end{cases}$$

Esempio ($\beta = 10, m = 5$):

$x = 9.3457881 \times 10^2$

- Troncamento: $fl(x) = 9.3457 \times 10^2$
- Arrotondamento: $fl(x) = 9.3458 \times 10^2$

Insieme dei Numeri di Macchina

Considerando base β, m cifre per la mantissa e t cifre per la caratteristica:

$\{0\} \cup \{x \in \mathbb{R} : x = \pm a_0 . a_1 \dots a_s \beta^b, s \leq m - 1, a_0 \neq 0, -(\beta^{t-1} -$

Soglie:

- **Overflow**: U (massimo numero rappresentabile)
- **Underflow**: L (minimo numero rappresentabile)

Errori di Rappresentazione

Definizioni

Dato x e la sua approssimazione $\tilde{x}$:

Errore Assoluto:

$$e_A = |x - \tilde{x}|$$

Errore Relativo:

$$e_R = \frac{|x - \tilde{x}|}{|x|}$$

Esempio Comparativo

Con $\beta = 10, m = 5$:

Caso 1: $x = 2.58954 \times 10^5, \tilde{x} = fl(x) = 2.5895 \times 10^5$

- $e_A = 4.0 \times 10^0$
- $e_R \approx c \times 10^{-5}$

Caso 2: $y = 2.58954 \times 10^{-2}, \tilde{y} = fl(y) = 2.5895 \times 10^{-2}$

- $e_A = 4.0 \times 10^{-7}$
- $e_R \approx c \times 10^{-5}$

Conclusioni:

- L'**errore assoluto** è influenzato dall'ordine di grandezza del dato
- L'**errore relativo** NON è influenzato dall'ordine di grandezza
- L'**errore relativo** fornisce indicazioni sull'approssimazione della mantissa, ovvero sull'**accuratezza** o **precisione**

Precisione di Macchina: εm

Definizione

La **precisione di macchina** ε_m è una limitazione superiore all'errore relativo:

$$\frac{|x - fl(x)|}{|x|} \leq \varepsilon_m \quad \forall x \neq 0$$

Valori

$$\varepsilon_m = \begin{cases} \beta^{1-m} & \text{TRONCAMENTO} \\ \frac{1}{2}\beta^{1-m} & \text{ARROTONDAMENTO} \end{cases}$$

Precisione Semplice e Doppia

Gli elaboratori utilizzano $\beta = 2$ e tecnica di arrotondamento.

Precisione Semplice (32 bits):

Componente	Bits
Segno del numero	1 bit
Mantissa	23 bits
Segno caratteristica	1 bit
Caratteristica	7 bits

Table 1: Distribuzione bits in precisione semplice

- $\varepsilon_m \approx 10^{-7}$
- $U \approx 10^{38}, L \approx 10^{-38}$

Precisione Doppia (64 bits):

Componente	Bits
Segno del numero	1 bit
Mantissa	52 bits
Segno caratteristica	1 bit
Caratteristica	10 bits

Table 2: Distribuzione bits in precisione doppia

- $\varepsilon_m \approx 10^{-16}$
- $U \approx 10^{308}, L \approx 10^{-308}$

Aritmetica Finita

Operazioni in Macchina

Poiché dati e risultati sono memorizzati con un numero finito di cifre, le operazioni non producono necessariamente numeri di macchina. Si introduce la notazione:

Operazione	Notazione	Significato
$x + y$	$x \oplus y$	$fl(fl(x) + fl(y))$
$x - y$	$x \ominus y$	$fl(fl(x) - fl(y))$
$x \times y$	$x \otimes y$	$fl(fl(x) \times fl(y))$
x/y	$x \oslash y$	$fl(fl(x)/fl(y))$

Table 3: Operazioni in aritmetica finita

Algoritmi per Operazioni in Macchina

Somma/Sottrazione:

1. Calcolare il floating di entrambi i numeri
2. Trasformare il numero con caratteristica minore in modo che abbiano la stessa caratteristica
3. Sommare le mantisse
4. Ricavare il floating del risultato (rinormalizzare)

Prodotto/Divisione:

1. Calcolare il floating di entrambi i numeri

2. Eseguire prodotto/divisione delle mantisse e sommare/sottrarre le caratteristiche
3. Ricavare il floating del risultato (rinormalizzare)

Proprietà delle Operazioni in Aritmetica Finita

Proprietà che valgono:

- ✓ **Proprietà commutativa**

Proprietà che NON valgono:

- ✗ **Proprietà associativa**
- ✗ **Proprietà distributiva**

Implicazione importante:

Formule o algoritmi matematicamente equivalenti (che producono lo stesso risultato in aritmetica esatta) possono produrre risultati diversi in aritmetica finita.

Esempio: Perdita della Proprietà Associativa

Con $\beta = 10, m = 4$, troncamento ($\varepsilon_m = 10^{-3}$):

$a = 2000, b = 2.5, c = 7.8$

(a ⊕ b) ⊕ c:

$(2.000 \times 10^3 \oplus 2.500 \times 10^0) \oplus 7.800 \times 10^0$

$= (2.000 \times 10^3 \oplus 0.002 \times 10^3) \oplus 7.800 \times 10^0$

$= 2.002 \times 10^3 \oplus 0.007 \times 10^3$

$= 2.009 \times 10^3$

a ⊕ (b ⊕ c):

$2.000 \times 10^3 \oplus (2.500 \times 10^0 \oplus 7.800 \times 10^0)$

$= 2.000 \times 10^3 \oplus 1.030 \times 10^1$

$= 2.000 \times 10^3 \oplus 0.010 \times 10^3$

$= 2.010 \times 10^3$

Risultato esatto: $(a + b) + c = a + (b + c) = 2010.3$

Conclusione: $(a \oplus b) \oplus c \neq a \oplus (b \oplus c)$

Errori nella Somma Algebrica

Analisi dell'Errore

Supponiamo (caso favorevole) che $x \oplus y = fl(x) + fl(y)$.

Definiamo:

$$\varepsilon_x = \frac{fl(x) - x}{x}, \quad \varepsilon_y = \frac{fl(y) - y}{y}$$

Quindi:

$$fl(x) = x(1 + \varepsilon_x), \quad fl(y) = y(1 + \varepsilon_y)$$

con $|\varepsilon_x| \leq \varepsilon_m$ e $|\varepsilon_y| \leq \varepsilon_m$.

Errore Relativo nella Somma

$$\frac{(x + y) - (fl(x) + fl(y))}{x + y} = -\frac{x}{x + y}\varepsilon_x - \frac{y}{x + y}\varepsilon_y$$

Problema critico:

Se $x \approx -y$, si può avere:

$$\frac{|(x + y) - (fl(x) + fl(y))|}{|x + y|} >> \varepsilon_m$$

anche se $|\varepsilon_x| \leq \varepsilon_m$ e $|\varepsilon_y| \leq \varepsilon_m$.

Questo fenomeno è noto come **cancellazione numerica** o **perdita di cifre significative**.

Algoritmo per il Calcolo di εm

Idea di Base

La precisione di macchina ε_m è la più piccola potenza della base β che viene "sentita" in macchina se sommata a 1.

Esempio ($\beta = 10, m = 8$, troncamento → $\varepsilon_m = 10^{-7}$):

- $fl(1 + 10^{-6}) = 1.000001 \times 10^0$
- $fl(1 + 10^{-7}) = 1.0000001 \times 10^0$
- $fl(1 + 10^{-8}) = 1 \times 10^0$ ← non viene "sentito"

Algoritmo

Se $fl(1 + \beta^{-q+1}) \neq 1$ ma $fl(1 + \beta^{-q}) = 1$, allora $\varepsilon_m = \beta^{-q+1}$.

1. Poni $p = 1$
2. Poni $p = \beta^{-1} \times p$
3. Poni $a = 1 + p$
4. Se $a > 1$ vai al passo 2. Altrimenti vai al passo 5
5. Poni $\varepsilon_m = \beta \times p$
6. Stampa ε_m

Conclusioni

Riepilogo dei Concetti Chiave

1. Si commettono **errori nella rappresentazione** dei numeri reali (errori di arrotondamento/troncamento)
2. Si commettono **errori nell'esecuzione delle operazioni** aritmetiche
3. Gli errori, anche se "piccoli", possono **accumularsi e amplificarsi** durante i calcoli
4. Le proprietà dell'aritmetica esatta (associatività, distributività) **non valgono** in aritmetica finita
5. È fondamentale **studiare la propagazione degli errori** per valutare i risultati degli algoritmi

Importanza dello Studio degli Errori

Idea tranquillizzante ma sbagliata:

"Poiché gli errori sono piccoli, anche i risultati sono affetti da errori piccoli."

Realtà:

Lo studio degli errori di arrotondamento e della loro propagazione è di **fondamentale importanza** per poter interpretare e valutare i risultati di qualunque algoritmo che operi con numeri reali.

La conoscenza della precisione di macchina ε_m è essenziale per:

- Valutare l'accuratezza dei risultati
- Impostare criteri di arresto appropriati negli algoritmi iterativi
- Scegliere algoritmi stabili e numericamente robusti
- Comprendere i limiti della computazione in aritmetica finita